

Algorithmes classiques vs algorithmes génétiques : analyse comparative

Problème concret : Planification d'emploi du temps

Scénario : Une université doit planifier 30 cours dans 12 salles avec 15 créneaux horaires, en respectant les contraintes des enseignants et les capacités des salles.

Approche classique : Recherche exhaustive

Algorithm 1: Algorithme classique naïf

Data: Liste des cours C , salles S , créneaux T
Result: Emploi du temps optimal
 $solutions \leftarrow \emptyset$;
foreach permutation p des affectations possibles **do**
 if estFaisable(p) **then**
 | $solutions.ajouter(p)$;
 end
end
if $solutions$ non vide **then**
 | **return** solution avec le moins de contraintes faibles;
end

Complexité et limitations

Nombre de combinaisons = $\prod_{i=1}^{30} (12 \times 15)^{30} \approx 10^{94}$ solutions possibles

- **Temps d'exécution** : Plusieurs fois l'âge de l'univers pour toutes les énumérer
- **Mémoire** : Nécessiterait des téraoctets pour stocker toutes les solutions
- **Réalisme** : Approche totalement impraticable malgré sa garantie de trouver l'optimum

Approche génétique : Exploration intelligente

Algorithm 2: Algorithme génétique pour le même problème

Data: Même problème, mais approche différente

Result: Solution quasi-optimale en temps raisonnable

$P \leftarrow \text{initialiserPopulation}(100 \text{ solutions aléatoires});$

for $g = 1$ **to** 100 **do**

foreach $ind \in P$ **do**

$\text{score}[ind] \leftarrow \text{évaluer}(ind);$

end

$P_{\text{meilleurs}} \leftarrow \text{sélectionner}(P, 30);$

$P_{\text{nouv}} \leftarrow P_{\text{meilleurs}};$

while $|P_{\text{nouv}}| < 100$ **do**

$e \leftarrow \text{croiser}(\text{choix aléatoire parmi } P_{\text{meilleurs}});$

$P_{\text{nouv}}.\text{ajouter}(e);$

end

$P \leftarrow P_{\text{nouv}};$

end

return meilleur de P ;

Comparaison directe

Critère	Algorithme classique	Algorithme génétique
Temps	$O(n!)$ ou exponentiel	$O(g \times p)$ (linéaire en générations)
Qualité	Optimum garanti	Quasi-optimum (90-99% optimal)
Résultat	Solution parfaite	Solution "assez bonne"
Temps réel	Des siècles	Quelques minutes
Usage mémoire	Énorme	Faible

TABLE 1 – Comparaison entre approche exhaustive et génétique

Quand privilégier chaque approche ?

Algorithme classique (exhaustif, programmation dynamique, etc.) :

- Problèmes avec peu de combinaisons ($n < 20$)
- Quand l'optimalité est critique (systèmes médicaux, aérospatial)
- Quand le temps de calcul n'est pas limité

Algorithme génétique :

- redCas où l'algorithme classique est naïf : $n > 50$ combinaisons

- Espace de recherche trop grand pour l'exploration exhaustive
- Quand on accepte une solution "suffisamment bonne"
- Problèmes avec plusieurs optima locaux
- **Exemples concrets** : planification, routage, design, apprentissage automatique

Conclusion

Pour notre problème de planification universitaire :

- **Algorithme classique** : Théoriquement correct, pratiquement inutilisable
- **Algorithme génétique** : Trouve une solution à 95% optimale en quelques minutes
- **Gain** : Passage de l'impossible au faisable